

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Software Testing & Quality Assurance

Software Testing Principles

- **Testing**

The execution of a program to find its faults.

- **Verification**

The process of proving the programs correctness.

Is the system correct with respect to some specification?

- **Validation**

The process of finding errors by executing the program in a real environment.

Did we build the right system?

- **Debugging**

Diagnosing the error and correct it.

What is Software Testing?

- **Several Definitions:**

- Testing is the process of establishing confidence that a program or system does what it is supposed to.
- Testing is the process of executing a program or system with the intent of finding errors.
- Testing can show the presence of bugs but never their absence.

What is Software Testing?

- The process of operating a system or component under specified conditions, observing or recording the results, and making an evaluation of some aspect of the system or component.
- A process of analyzing a software item to detect the differences between existing and required conditions (that is defects / errors / bugs) and to evaluate the features of the software item.

Goal of Testing

- 1) Finding faults in the software
- 2) Demonstrating that there are no faults in the software (for the test cases that has been used during testing).
- 3) To Reduce Costly Errors
 - It is not possible to *prove* that there are no faults in the software using testing.

Why To Reduce Costly Errors Example

- In May 1992, Pepsi ran a promotion in the Philippines. It told customers they could win a million pesos (approx. \$40,000) if they bought a bottle of Pepsi and found number 349 stamped on the underside of the bottle cap.
- Unfortunately, due to a software error, 800,000 bottle caps were produced with number 349 instead of one, which was an equivalent of \$42 billion in prize money. It cost the company dearly as some people pursued their claims through the courts and Pepsi paid out millions of dollars in compensation.

Basic Definitions

- **What is an error?**

- Error is deviation from actual and expected value.
- It represents mistake made by people.

- **What is a fault?**

- Fault is incorrect step, process or data definition in a computer program which causes the program to behave in an unintended or unanticipated manner.
- It is the result of the error.
- We might speak of faults of commission and faults of omission.

Basic Definitions

- **What is a bug?**

- Bug is a fault in the program which causes the program to behave in an unintended or unanticipated manner.
- It is an evidence of fault in the program.

- **What is a failure?**

- Failure is the inability of a system or a component to perform its required functions within specified performance requirements.
- Failure occurs when fault executes.
- This definition relates failures only to faults of commission

Basic Definitions

- **What is a defect?**

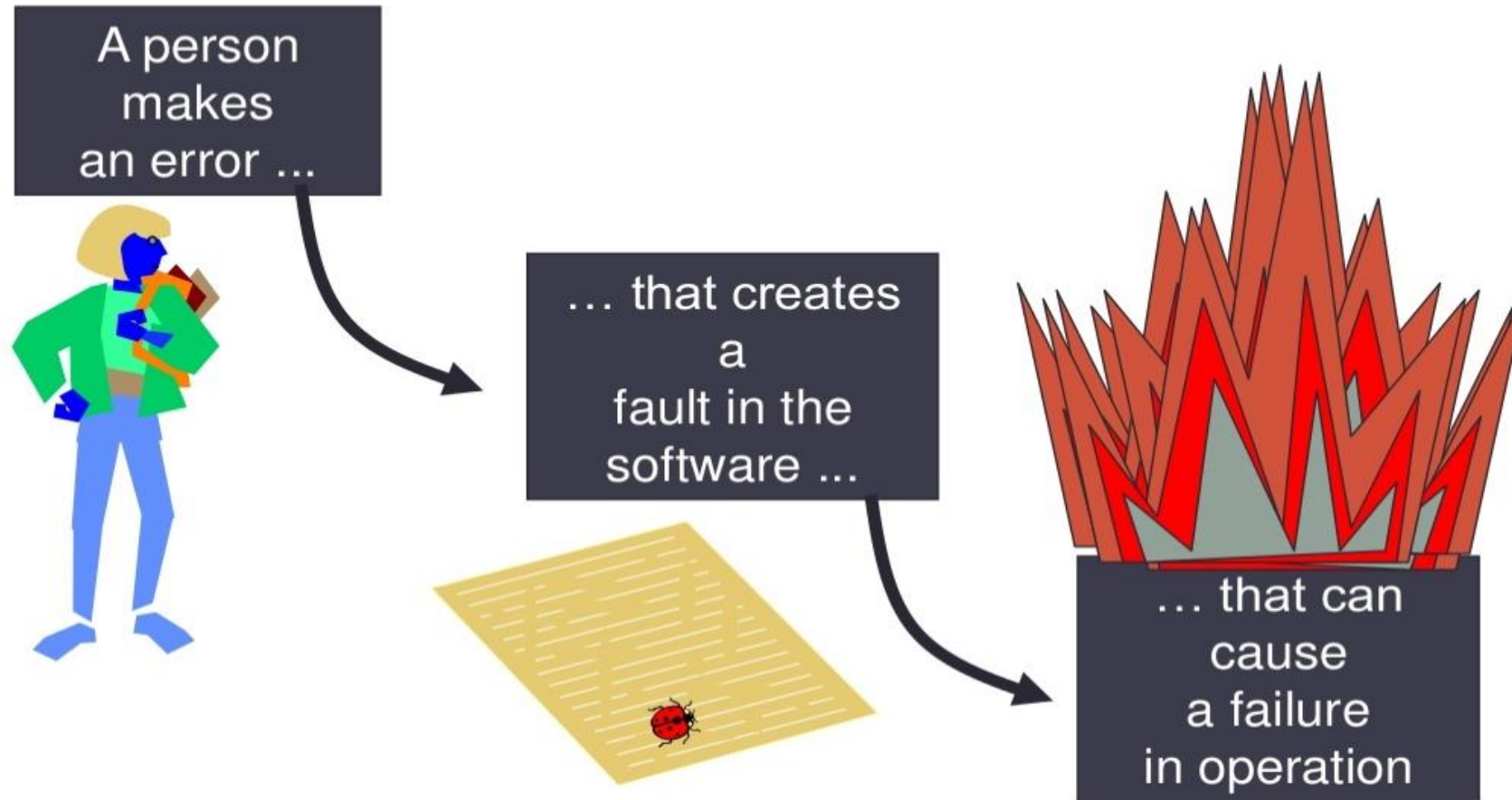
- Defect is an error in coding or logic that causes a program to malfunction or to produce incorrect/unexpected results.
- A defect is said to be detected when a failure is observed.

- **What is a Incident?**

- When a failure occurs, it may or may not be readily apparent to the user (or customer or tester)
- An incident is the symptom associated with a failure that alerts the user to the occurrence of a failure.

Error - Fault - Failure

Error - Fault - Failure



IEEE Definitions

- **Failure:** External behavior is incorrect.
- **Fault:** Discrepancy in code that causes a failure.
- **Error:** Human mistake that caused fault.
- **Note:**
- **Error** is terminology of Developer.
- **Bug** is terminology of Tester.

Error Example

- **Error:** An error is a mistake, misconception, or misunderstanding on the part of a software developer.

1.	If (a=b)	
2.	{	
3.	c=a/b(;	//Error
4.	}	

If programmer missed the semicolon at the end of 3rd line of code and tries to compile, then an error message is occurred. This type of mistake called ERROR.

Bug Example

- **Bug:** When an issue, unexpected result or deviation in actual functionality is found out by an Author (who wrote the code) after compilation of program and during any testing phase (At during development, Beta version or at production) is called Bug.

Author used the variable 'd' instead of variable 'b' or use divide symbol instead of plus symbol, due to this there is no compilation error is occurred but it produce some unaccepted result.

Defect Example

- **Defects:** When an issue, unexpected result or deviation in actual functionality is found out by a moderator (Not an Author of code) during any phase (At during development, Beta version or at production sites) of testing is called Defect.

Author used the variable 'd' instead of variable 'b' or use divide symbol instead of plus symbol, due to this there is no compilation error is occurred but it produce some unaccepted result but here issue is found out by Moderator.

Fault Example

- **Fault:** it is a state during testing, which indicate deviation from actual functionality.

In this example, Author used the 'b' instead of 'd', this state when we got a deviation from actual result. This state is called Fault.

Failure Example

- **Failure:** It is a state which rose due to a fault and affected a single part of functionality.

Due to this Fault state, any other single line code or single module is affected, this state is called Failure.

Complete Example (add two numbers)

```
#include <iostream>
using namespace std;
int main()
{
    int firstNumber;           //Declare a variable
    int secondNumber;
    int sum;
    firstNumber=5;
    secondNumber=3;
    sum=firstmNumber - secondNumber; //Bug
    cout<<"The Sum of 5+3 ="<<sum<<endl;
    return 0;
}
```

Result of this Program is

When you compile and run this program you see the printed statement as below:

The sum of $5+3= 2$

So after compiling and running this program we realize the program has failed to do what it was supposed to do.

The program was supposed to add two numbers but it certainly did not add 5 and 3. $5 + 3$ should be 8, but the result is 2. There could be various reasons as to why the program displays the answer 2 instead of 8. For now we have detected a failure.

As the failure has been detected a defect can be raised.

Now lets go back to the program and analyze what was the fault in the program.

Basic Definitions

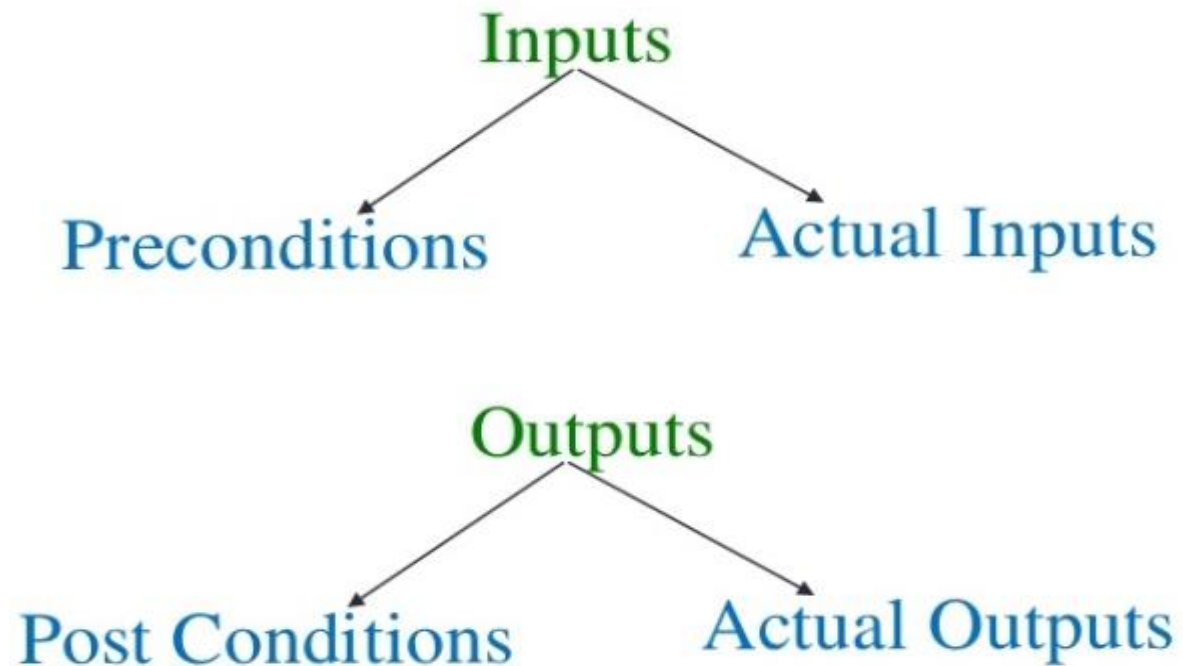
- Testing is obviously concerned with errors, faults, failures, and incidents.

"A test is the act of exercising software with test cases"

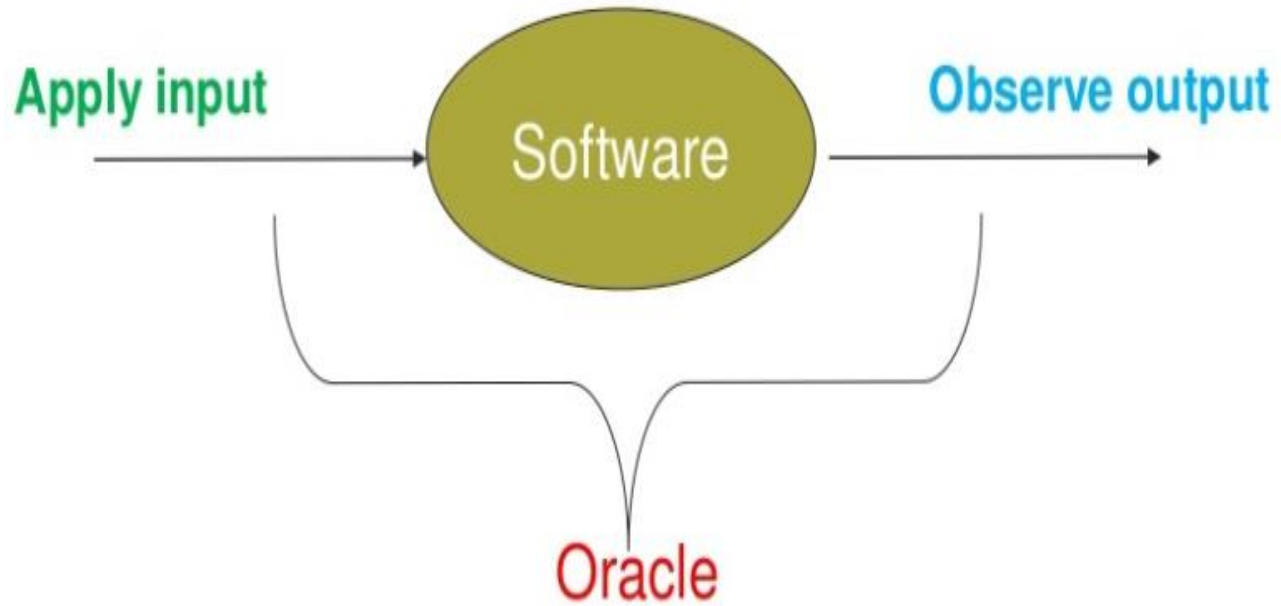
- A test has two distinct goals:
 - 1) To find failures or
 - 2) Demonstrate correct execution

Basic Definitions- Test Case

- A test case has an identity and is associated with a program behavior.
- It has a set of inputs and a list of expected outputs.



Testing



Validate the observed output against the expected output
Is the observed output the same as the expected output?

Writing Good Test Cases

- As far as possible, write test cases in such a way that you test only one thing at a time. Do not overlap or complicate test cases. Attempt to make your test cases 'atomic'.
- Ensure that all positive scenarios and negative scenarios are covered.
- **Characteristics of a good test case:**
 - *Accurate*: Exacts the purpose.
 - *Economical*: No unnecessary steps or words.
 - *Traceable*: Capable of being traced to requirements.
 - *Repeatable*: Can be used to perform the test over and over.
 - *Reusable*: Can be reused if necessary.

The Test Plan

The Test Plan

- who
- what
- when
- where
- how



Who Tests the Software?



Developer

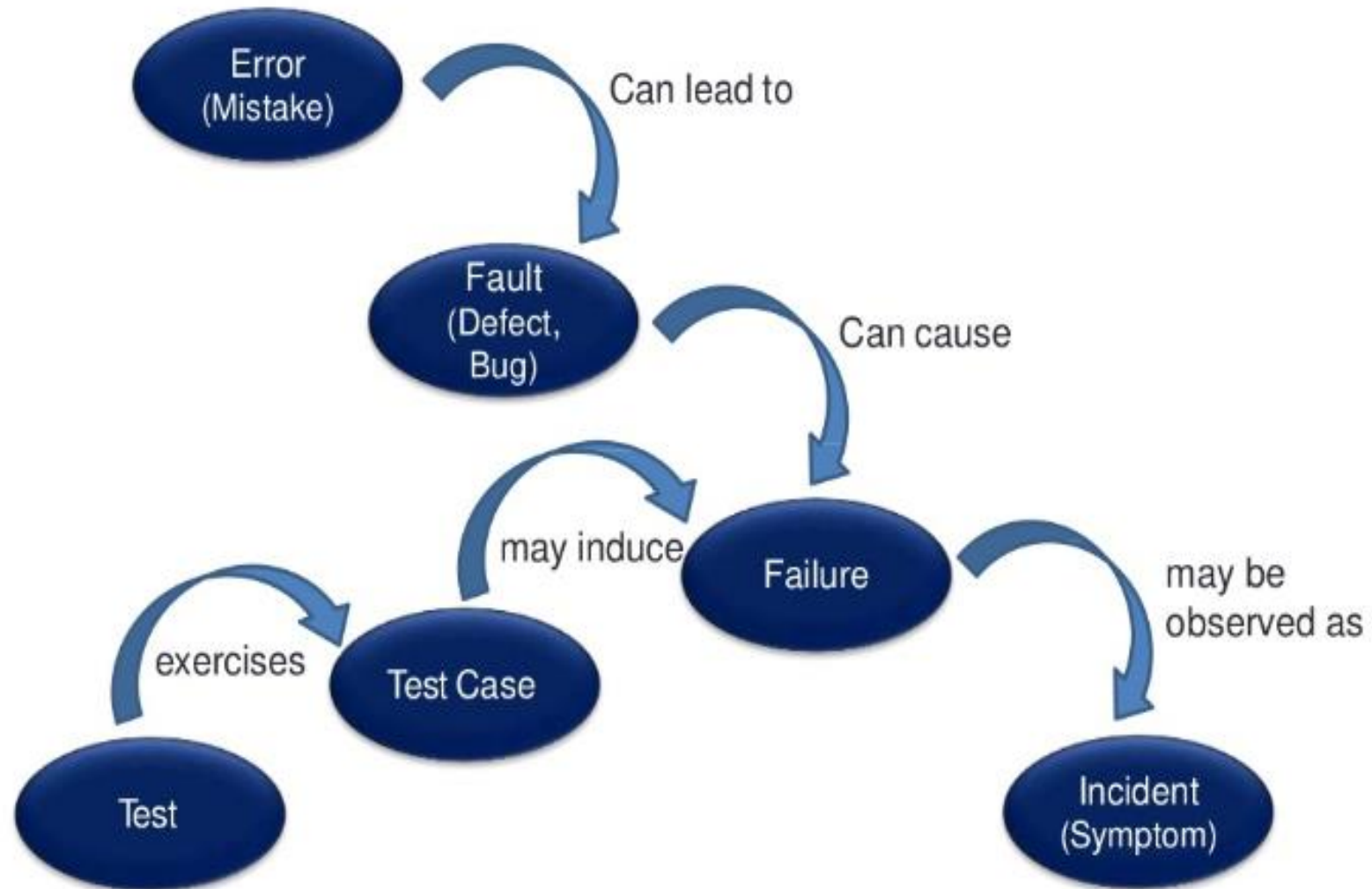
Understands the system
but, will test "gently"
and, is driven by "delivery"



Independent Tester

Must learn about the system,
but, will attempt to break it
and, is driven by quality

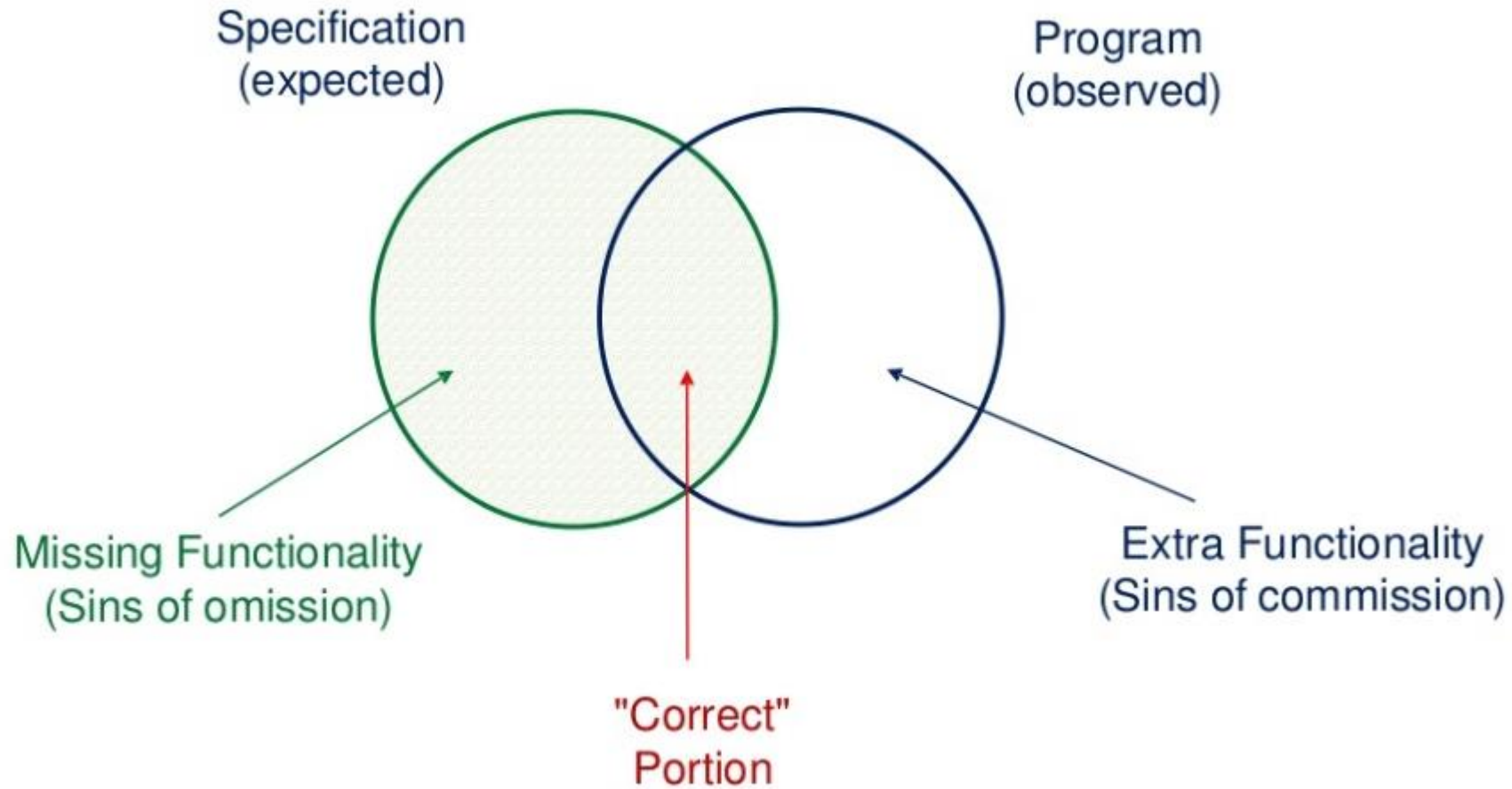
The Tester's Language



Insights from a Venn Diagram

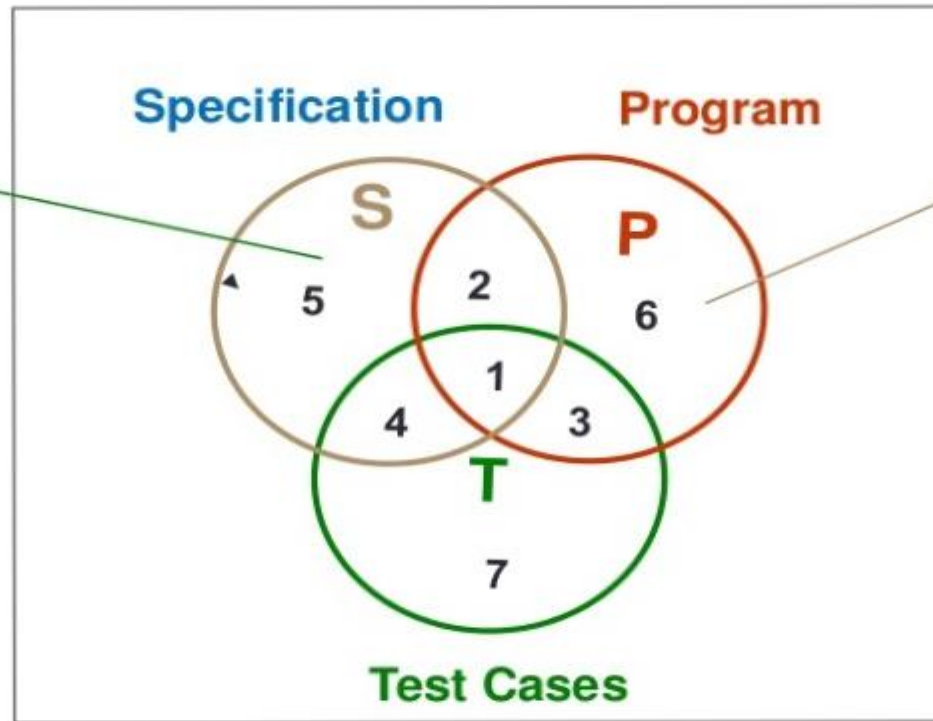
- Testing is fundamentally concerned with behavior, and behavior is orthogonal to the code-based view common to software (and system) developers
- A quick distinction is that:
 - The **code-based view** focuses on what it is
 - The **behavioral view** considers what it does

Program Behaviors



faults of omission

What if there are specified behaviors that have not been programmed?

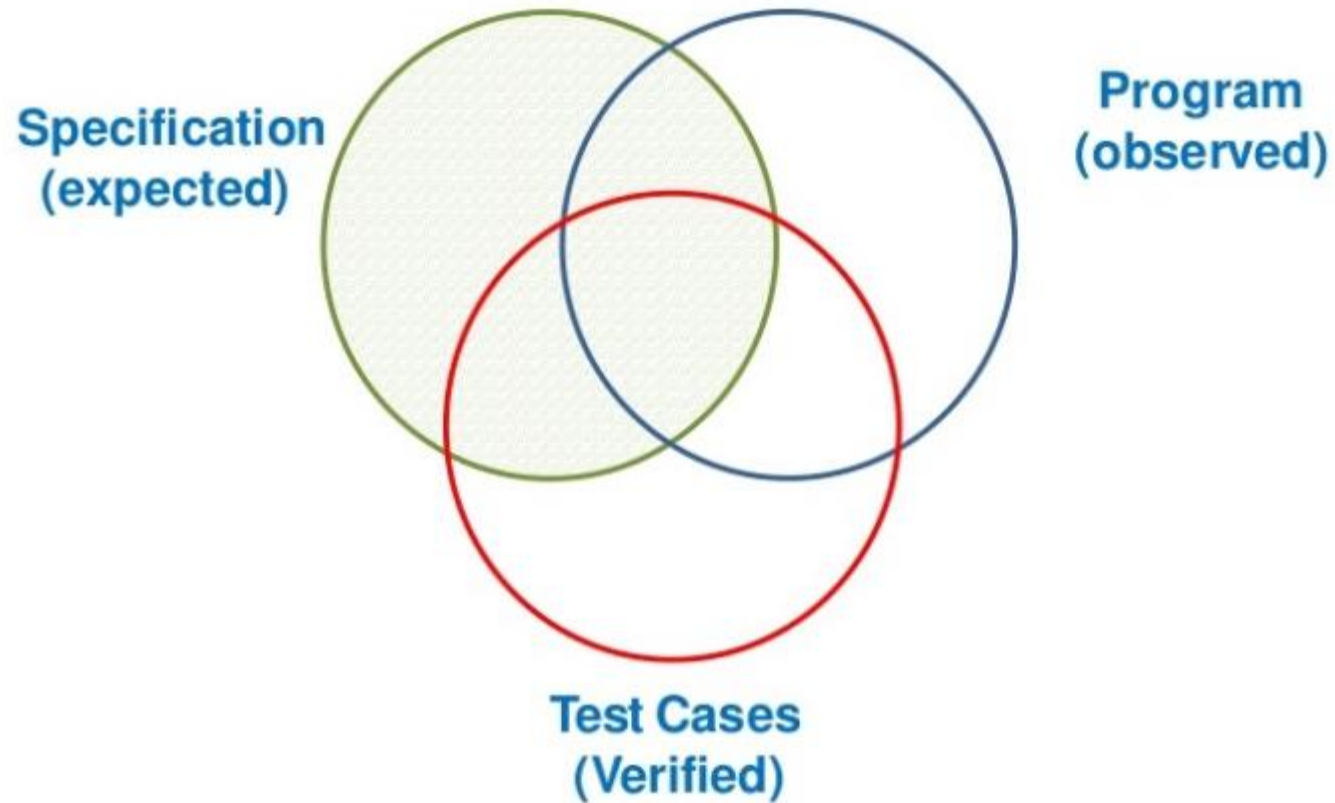


faults of commission

what if there are programmed (implemented) behaviors that have not been specified?

- A very good view of testing is that it is the determination of the extent of program behavior that is both specified and implemented
- What can a tester do to make the region where these sets all intersect be as large as possible?
- How the test cases in the set T are identified?

Program Behavior and Test Cases



Common Verification and Validation Activities

- **Static Analysis Techniques**

- Based solely on the (manual or automated) examination of project documentation of software models and code, and of other related information about requirements and design
- Generally yields valid results, but may be weak in precision

- **Dynamic Test Techniques**

- Exercise the software in order to expose possible failures
- Behavioral and performance properties of the program are also observed
- Yields more precise results, but only holding for the examined executions

Static Techniques

- Can be applied at the requirements phase; at the design phase; and during the implementation phase
- Traditional Static Techniques - heavily manual, error-prone, time consuming
 - **Software inspection** - The step-by-step analysis of the documents (deliverables) produced, against a compiled checklist of common and historical defects
 - **Software reviews** - The process by which different aspects of the work product are presented to project personnel (managers, users, customer etc) and other interested stakeholders for comment or approval
 - **Code reading** - The desktop analysis of the produced code for discovering typing errors that do not violate style or syntax
 - **Algorithm analysis and tracing** - The process in which the complexity of algorithms employed and the worst case, average-case and probabilistic analysis evaluations can be derived
- Static Analysis Techniques relying on use of Formal Methods

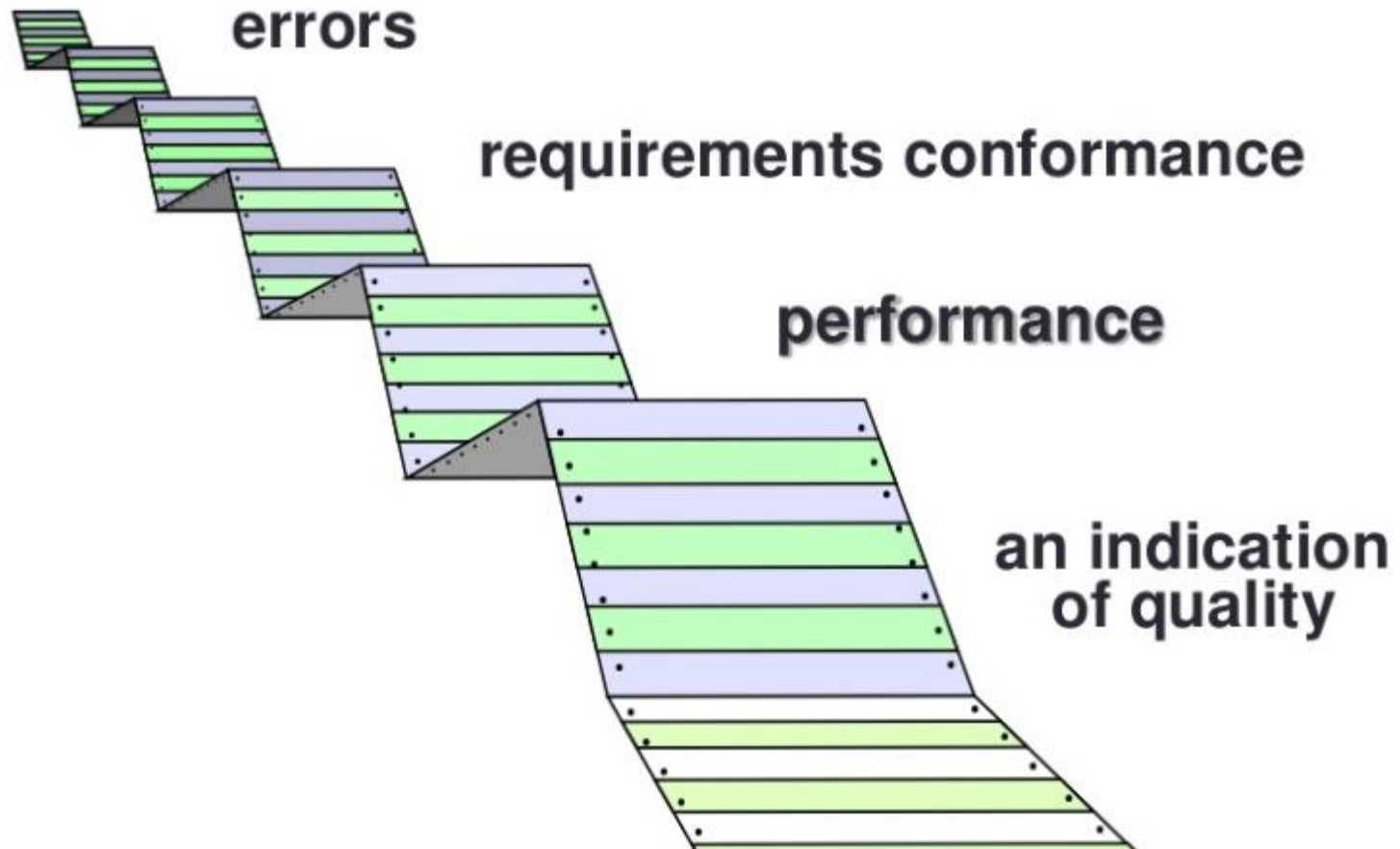
Dynamic Techniques

- Dynamic Techniques

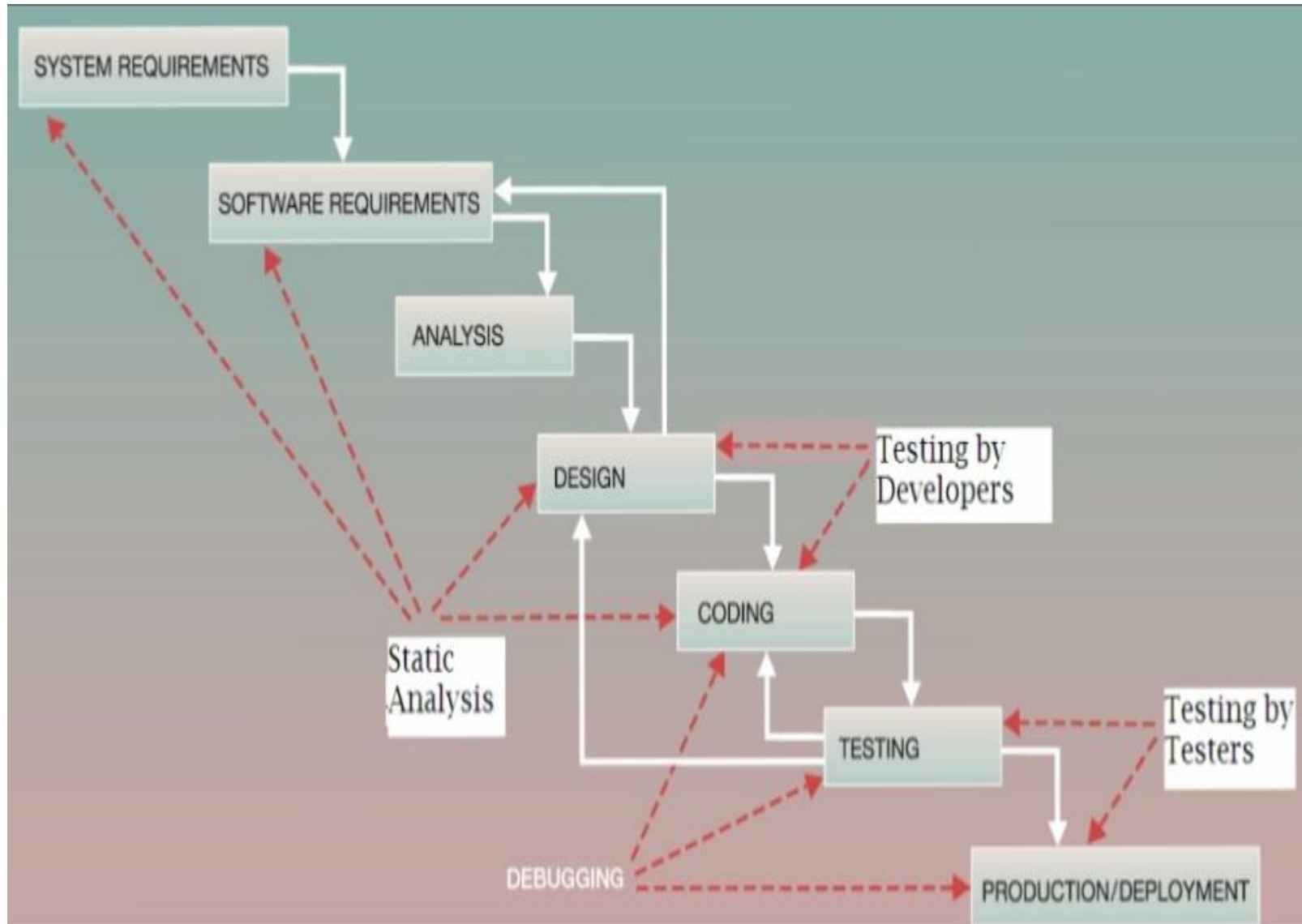
- **Testing** - Based on the execution of the code on valued inputs (definition of the parameters and environmental conditions that characterize a system state must be included when necessary)

- **Profiling** - A program profile records the number of times some entities of interest occur during a set of controlled executions

What Testing Shows



V&V, Testing, Debugging and Life



Testing a Software



Requirements Definition
Requirements Specification



Functional Requirements
Nonfunctional Requirements



Design Specification



Code = System

Error and Fault Taxonomies

Process versus Product

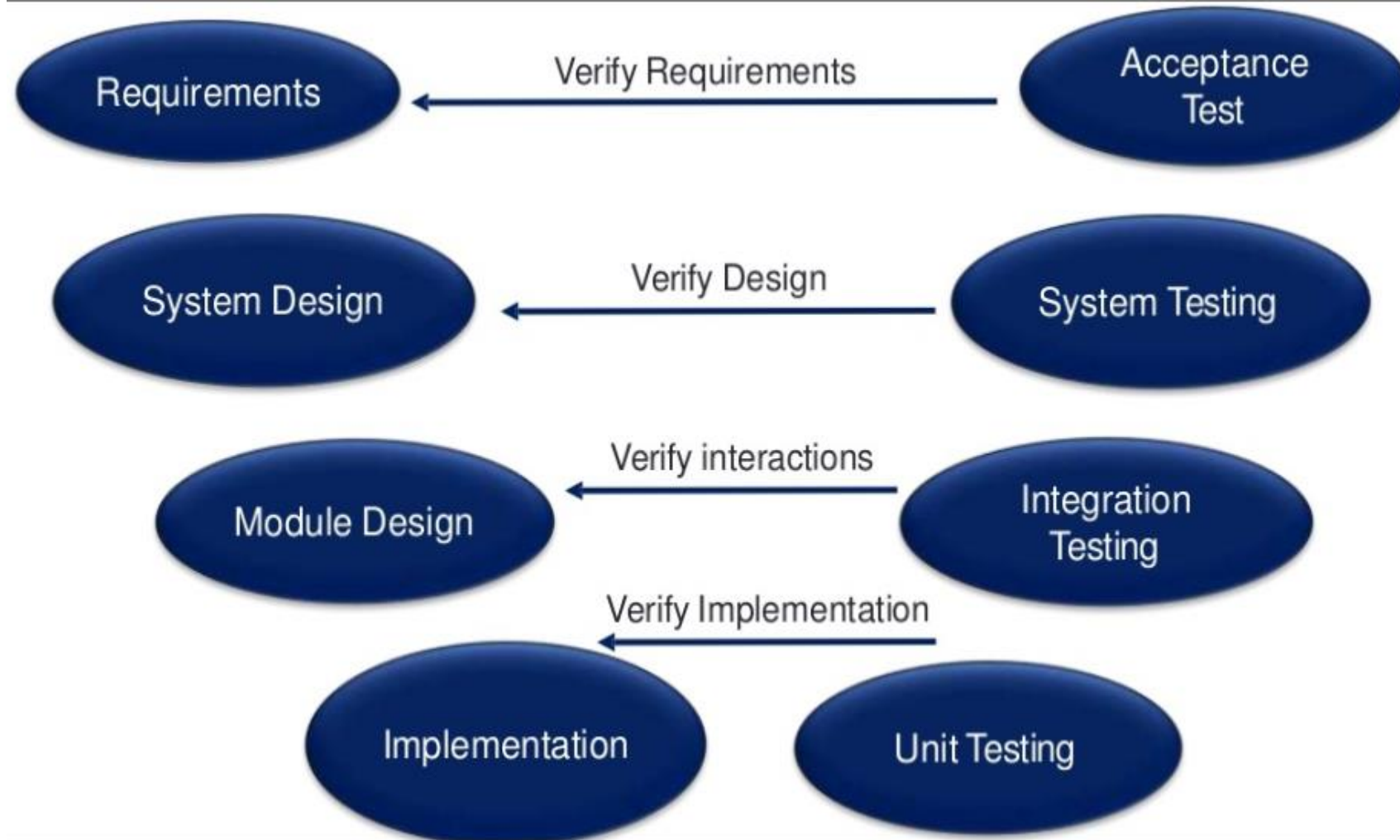
- process refers to how we do something, and
- product is the end result of a process

SQA is more concerned with reducing errors endemic in the development process, while testing is more concerned with discovering faults in a product.

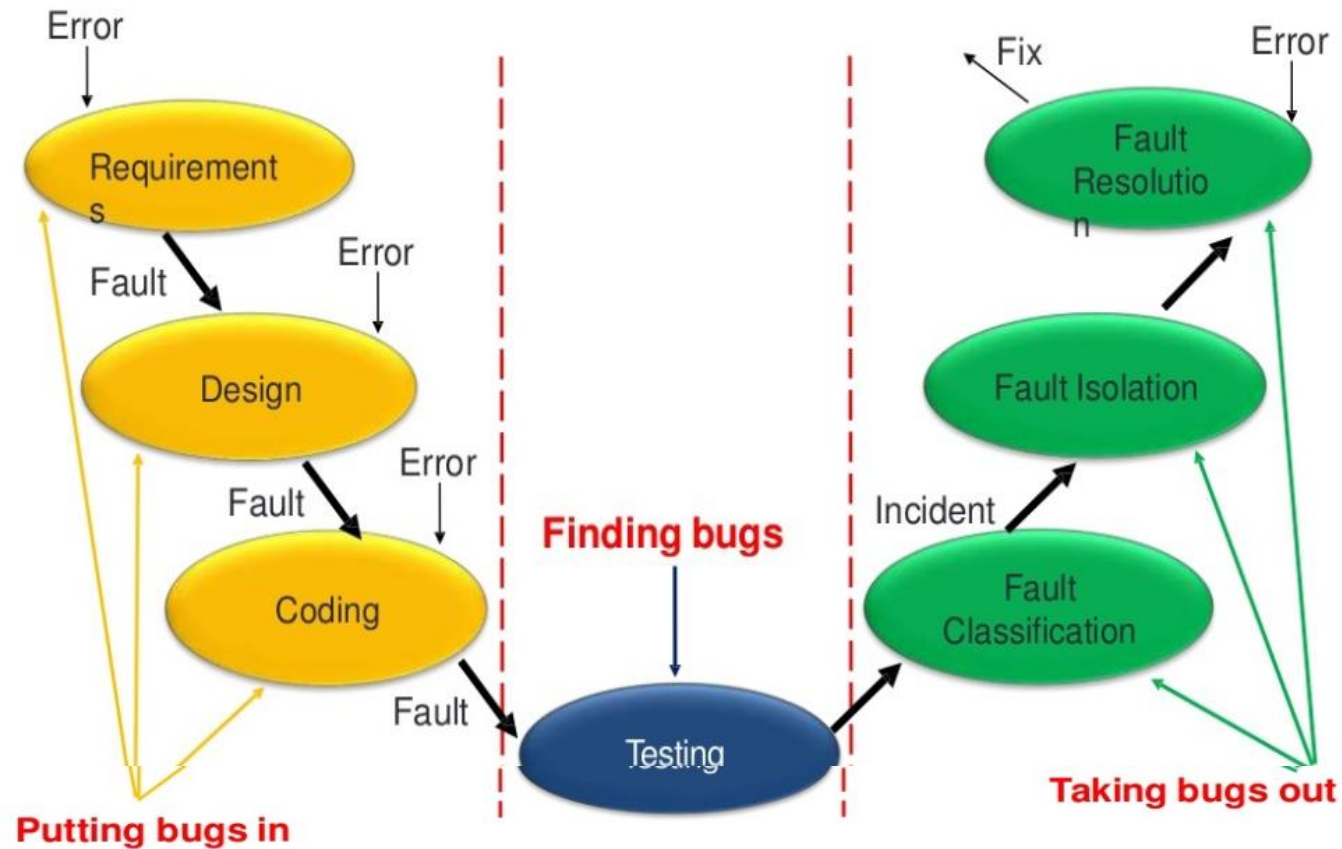
Faults can be classified in several ways:

- the development phase where the corresponding error occurred,
- the consequences of corresponding failures,
- difficulty to resolve,
- risk of no resolution, and so on.

A Software Life-Cycle Model



A Testing Life-Cycle Model



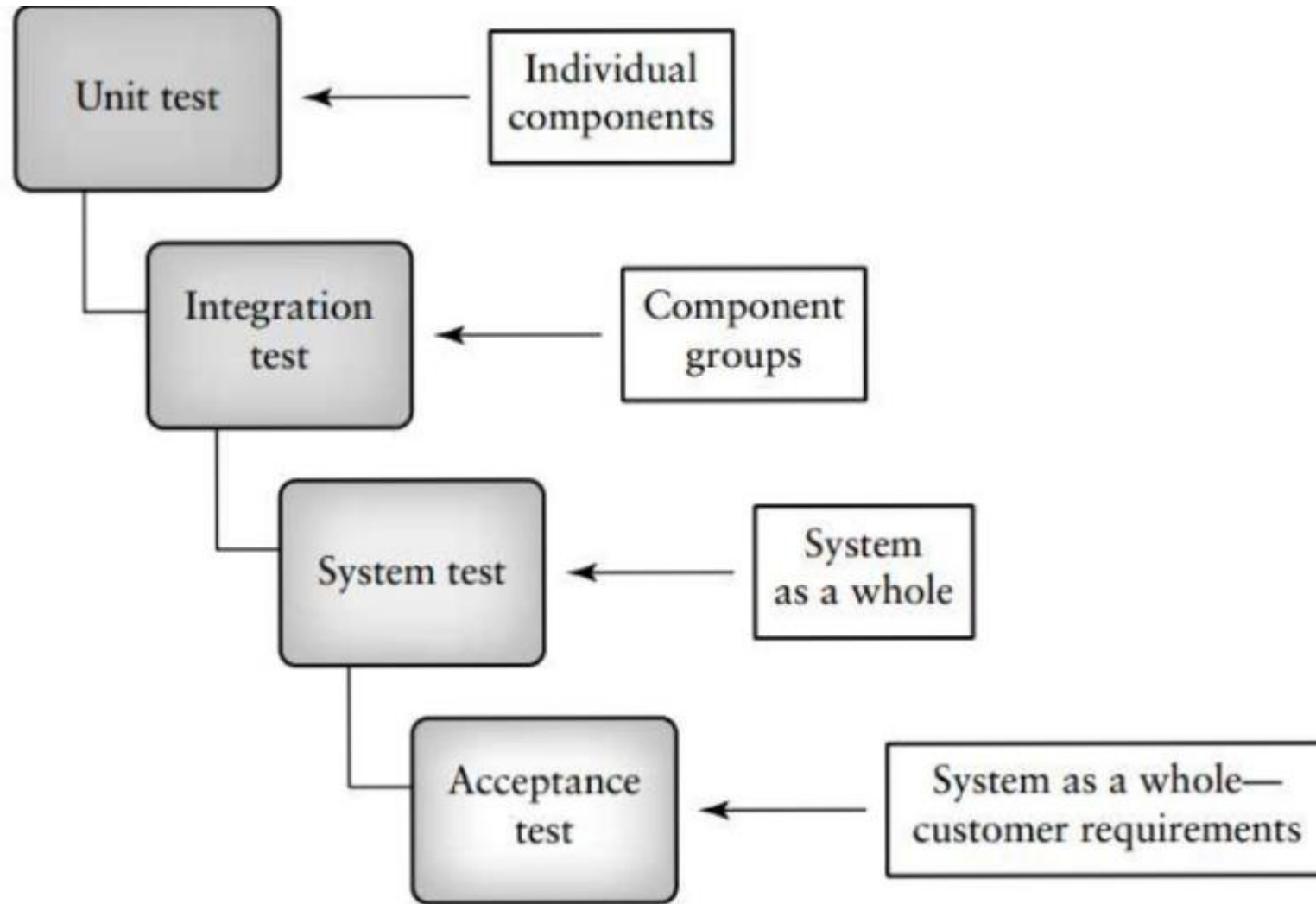
Test Design



Testing Level

- Unit Testing
- Integration Testing
- Acceptance Testing
- System Testing
- Regression Testing

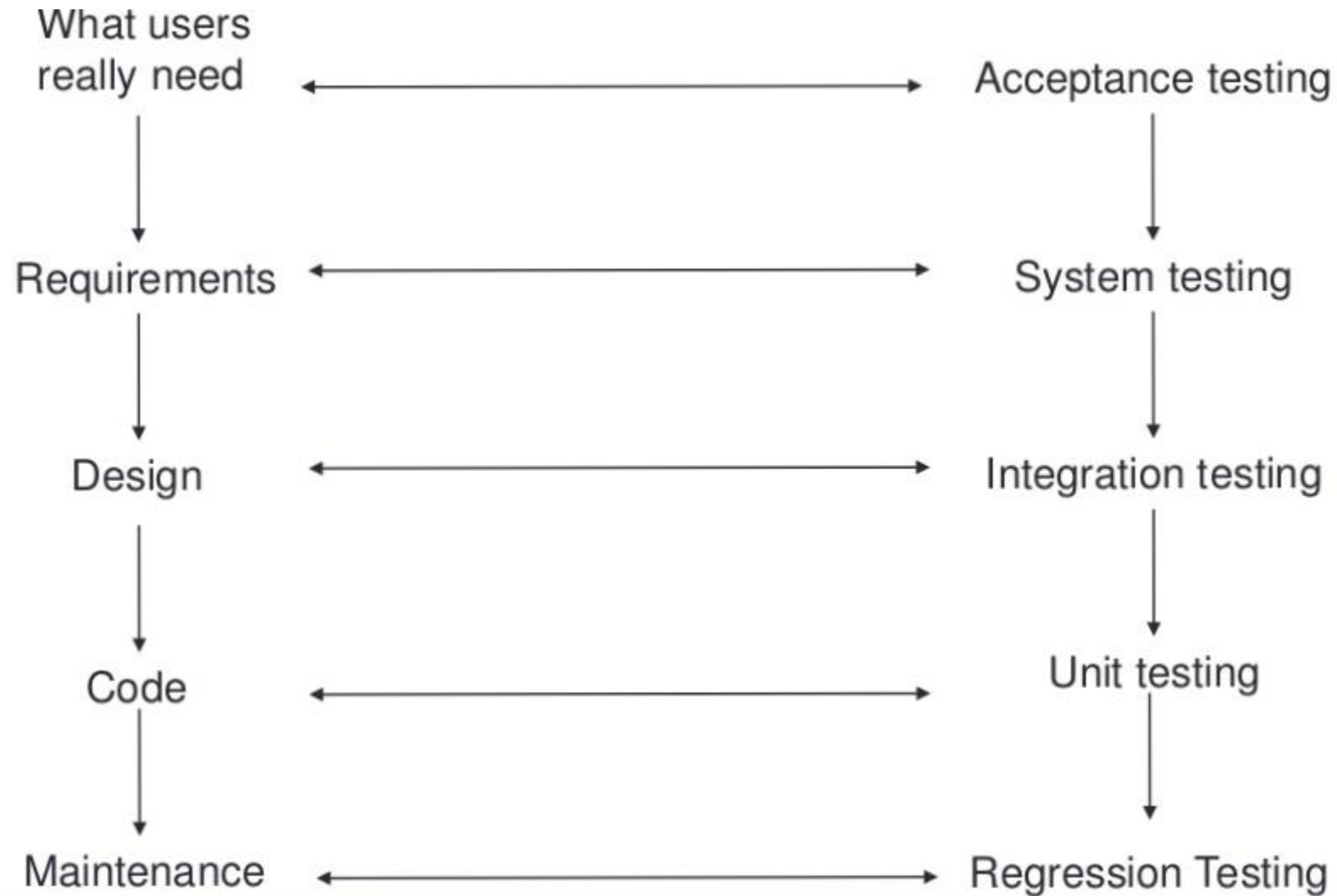
Testing Level



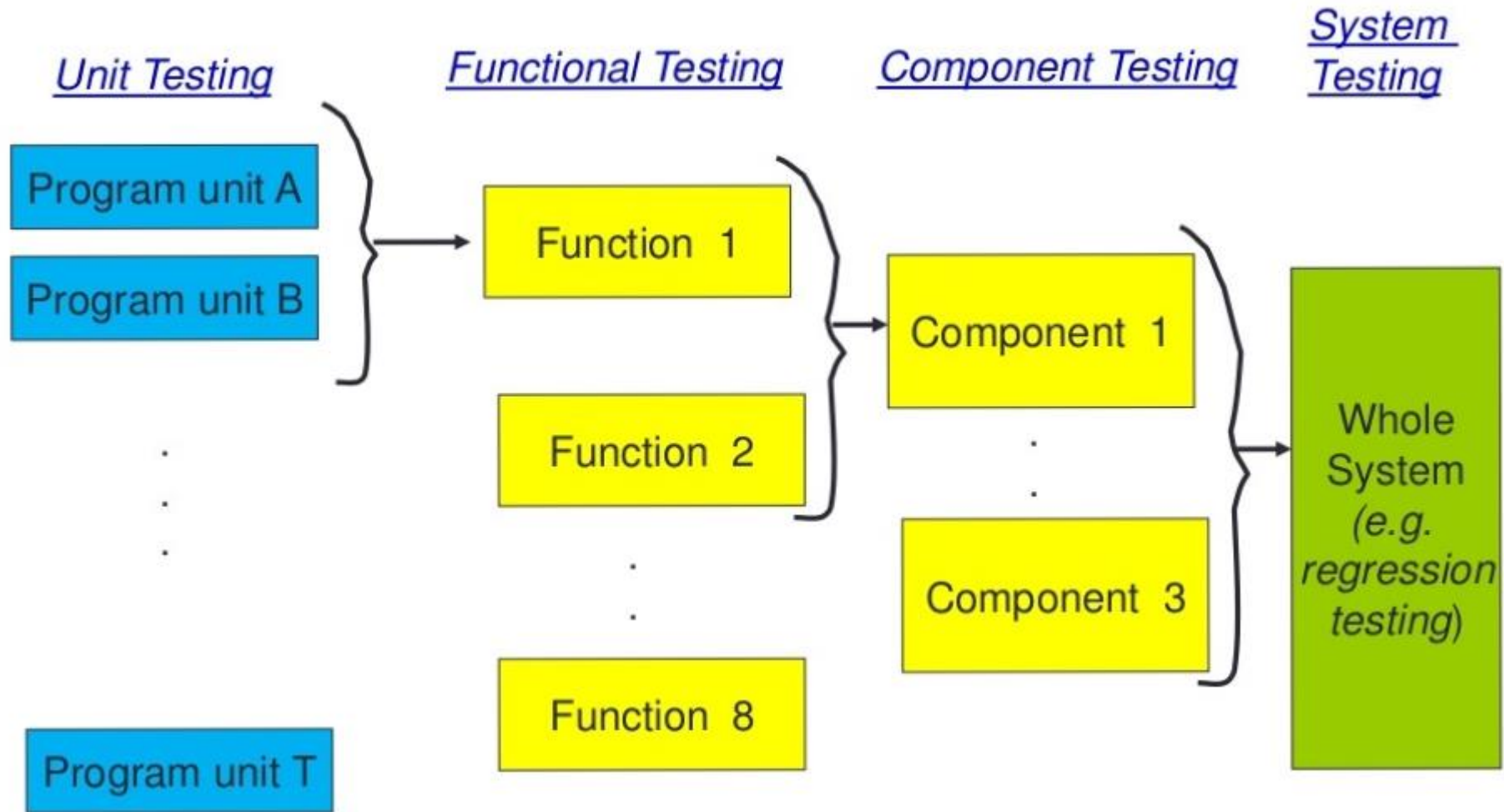
Testing Level

- **Unit testing:** checking the behavior of single modules
- **Integration testing:** checking the behavior of module cooperation.
- **Acceptance testing:** the software behavior is compared with end user requirements
- **System testing:** the software behavior is compared with the requirements specifications
- **Regression testing:** to check the behavior of new releases

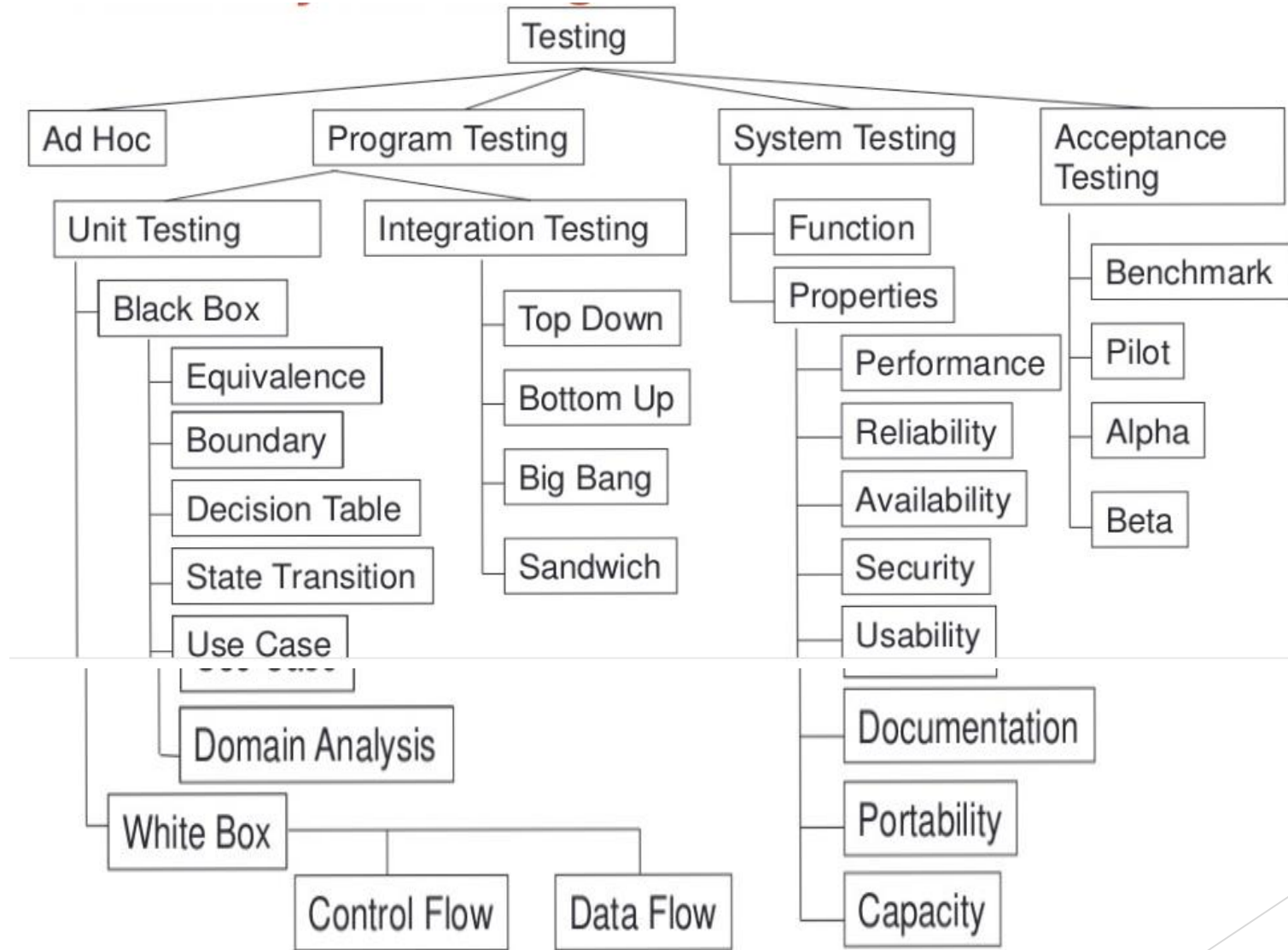
Levels of Testing



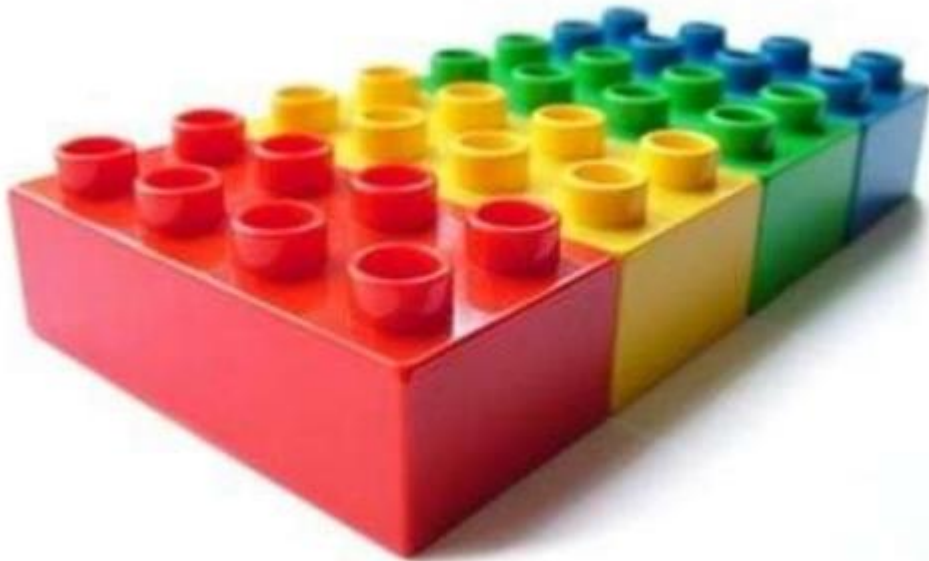
Different Levels of Testing



Hierarchy of Testing



Unit Testing



What is a Unit?

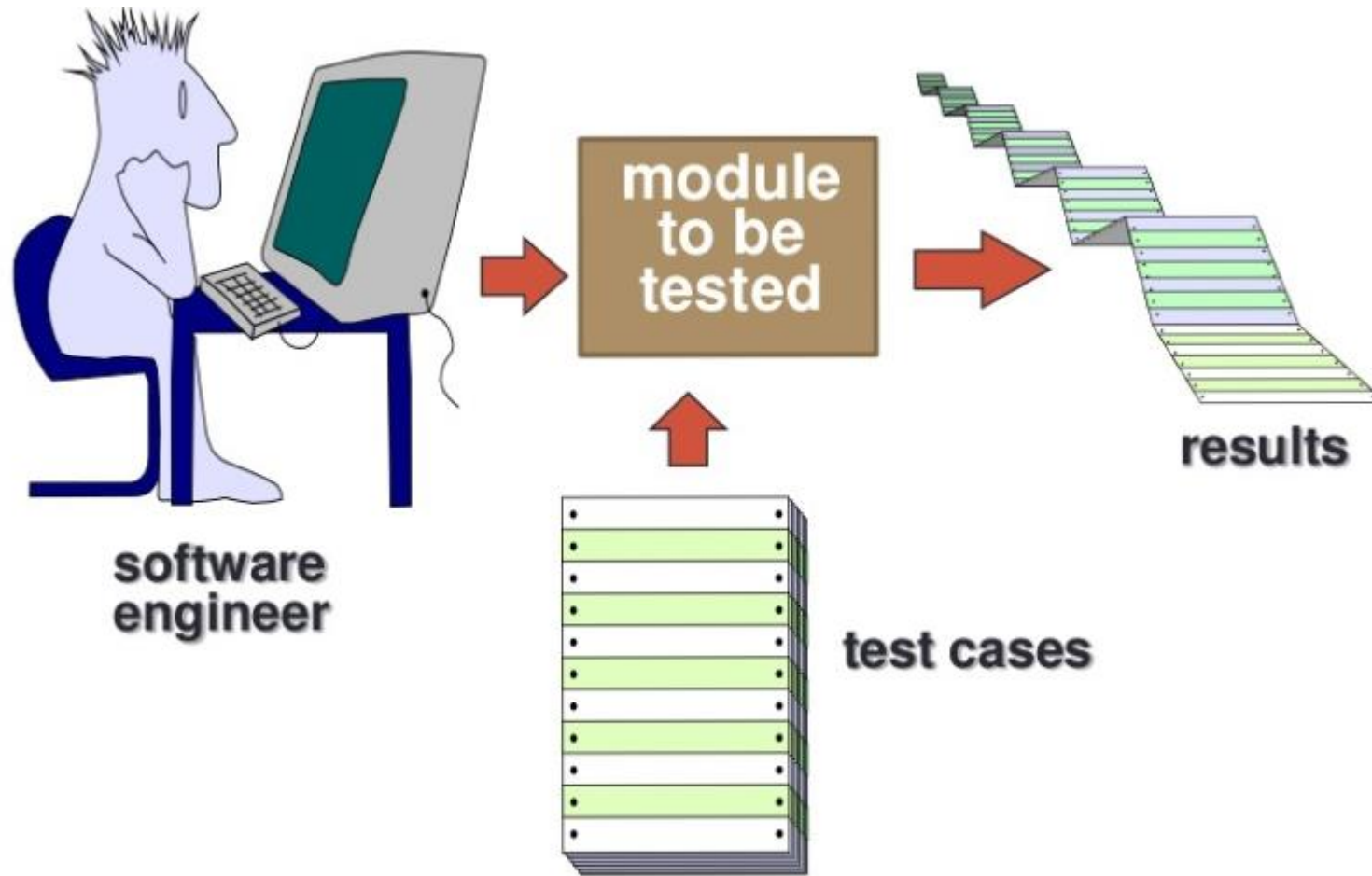
- The smallest piece of code that can be tested in isolation

```
public class MyMath
{
    public int    Add(int i, int j)
    {
        return i + j;
    }
}
```

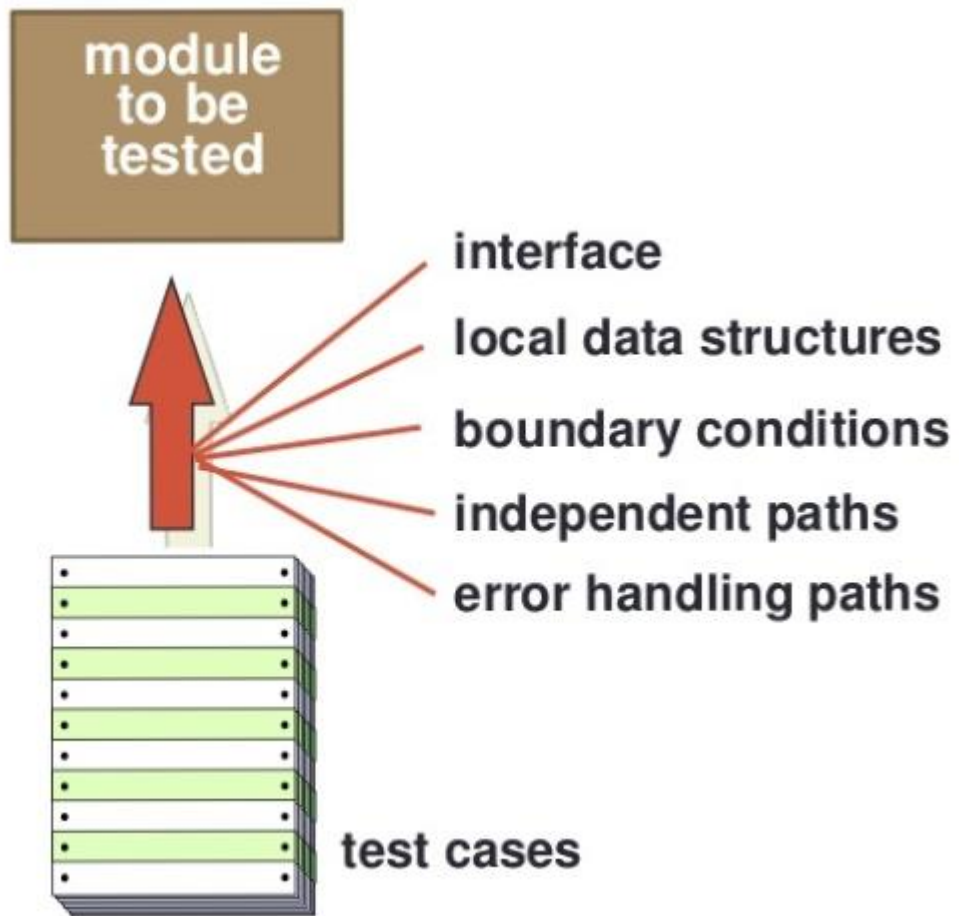
What is Unit Test?

- Unit Testing is a level of the software testing process where individual units/components of a software/system are tested.
- The purpose is to validate that each unit of the software performs as designed.
- Concerned with functional correctness and completeness of individual program units.
- Its goal is to isolate each part of the program and show that the individual parts are correct.

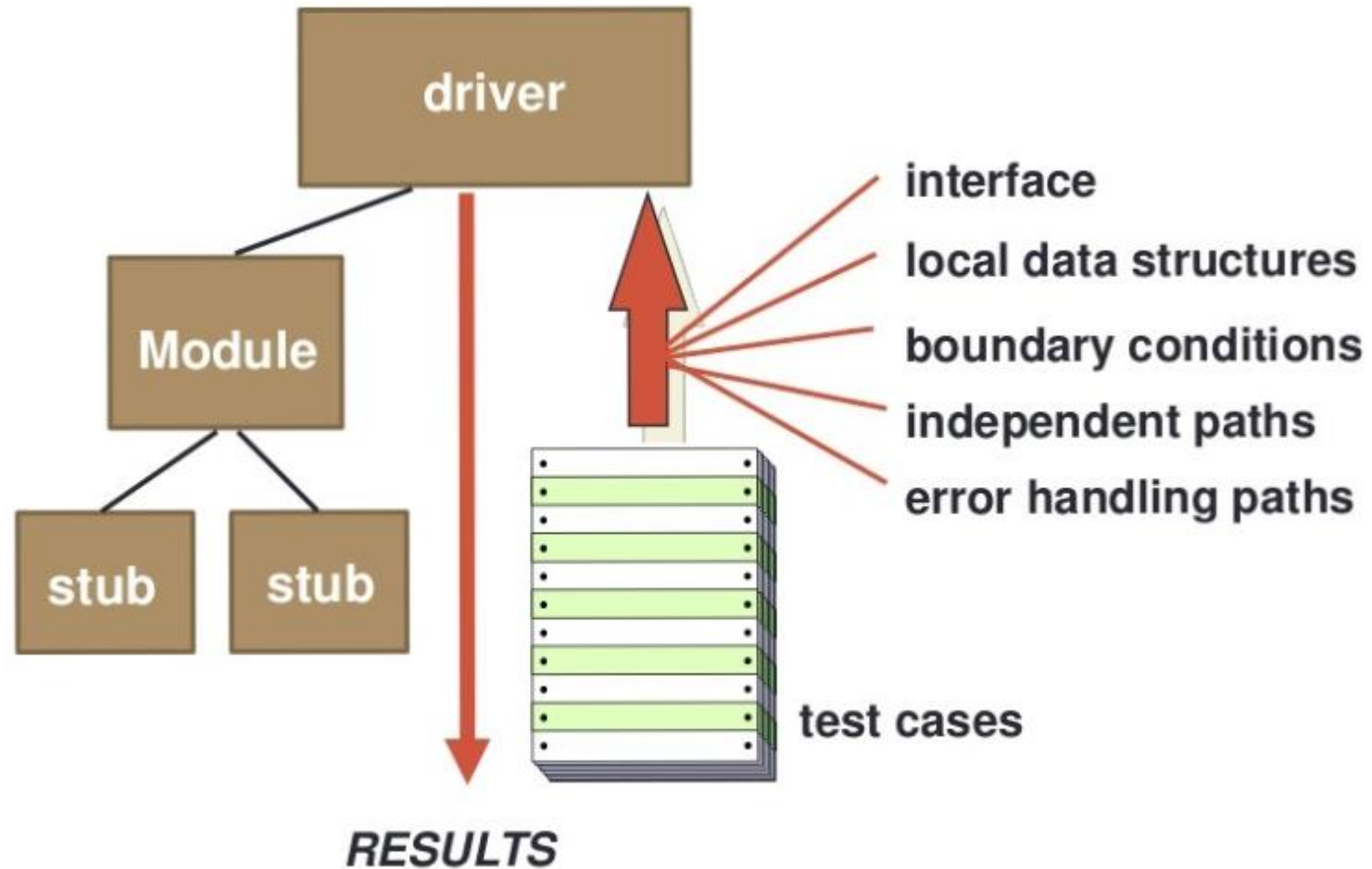
Unit Testing



Unit Testing



Unit Testing

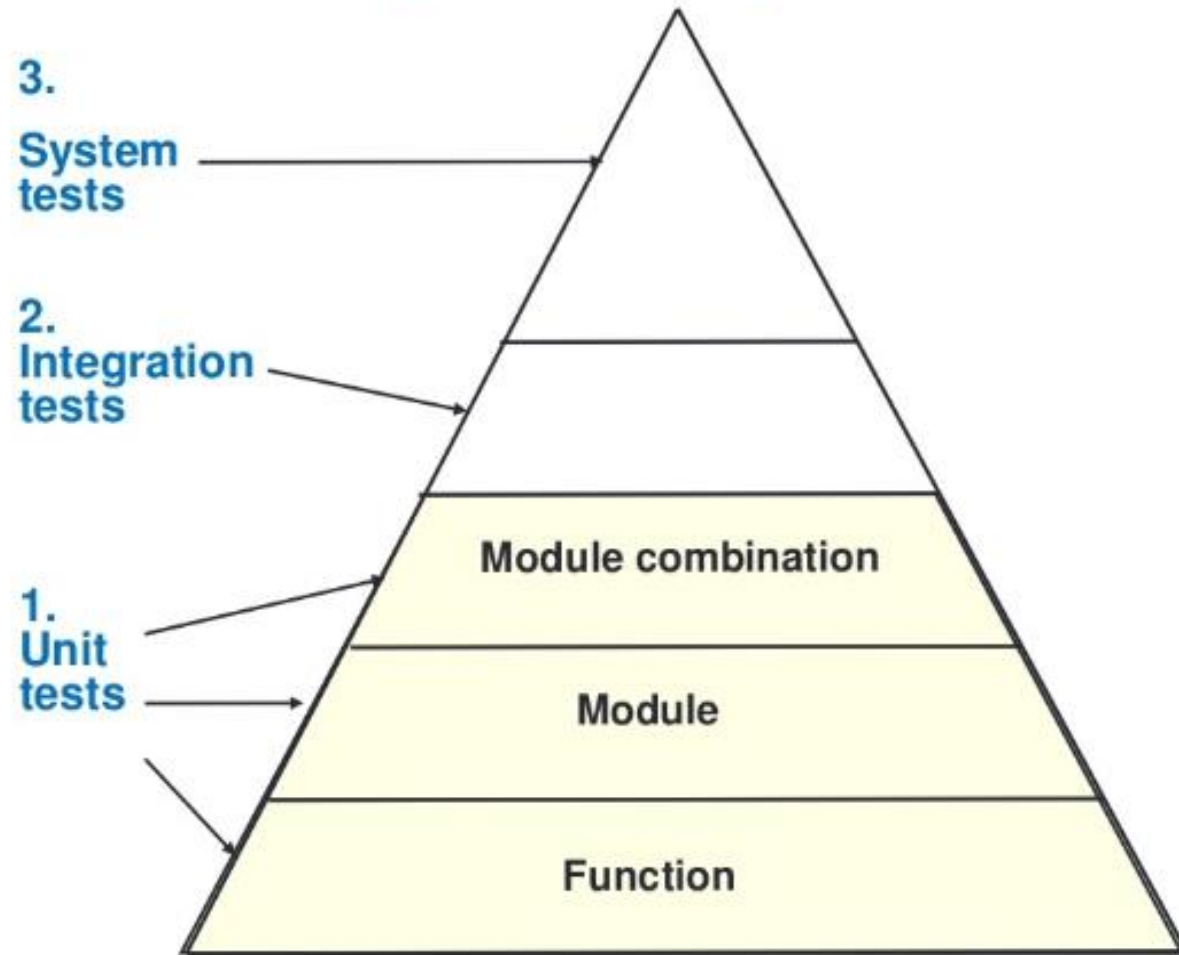


What is Unit Test?

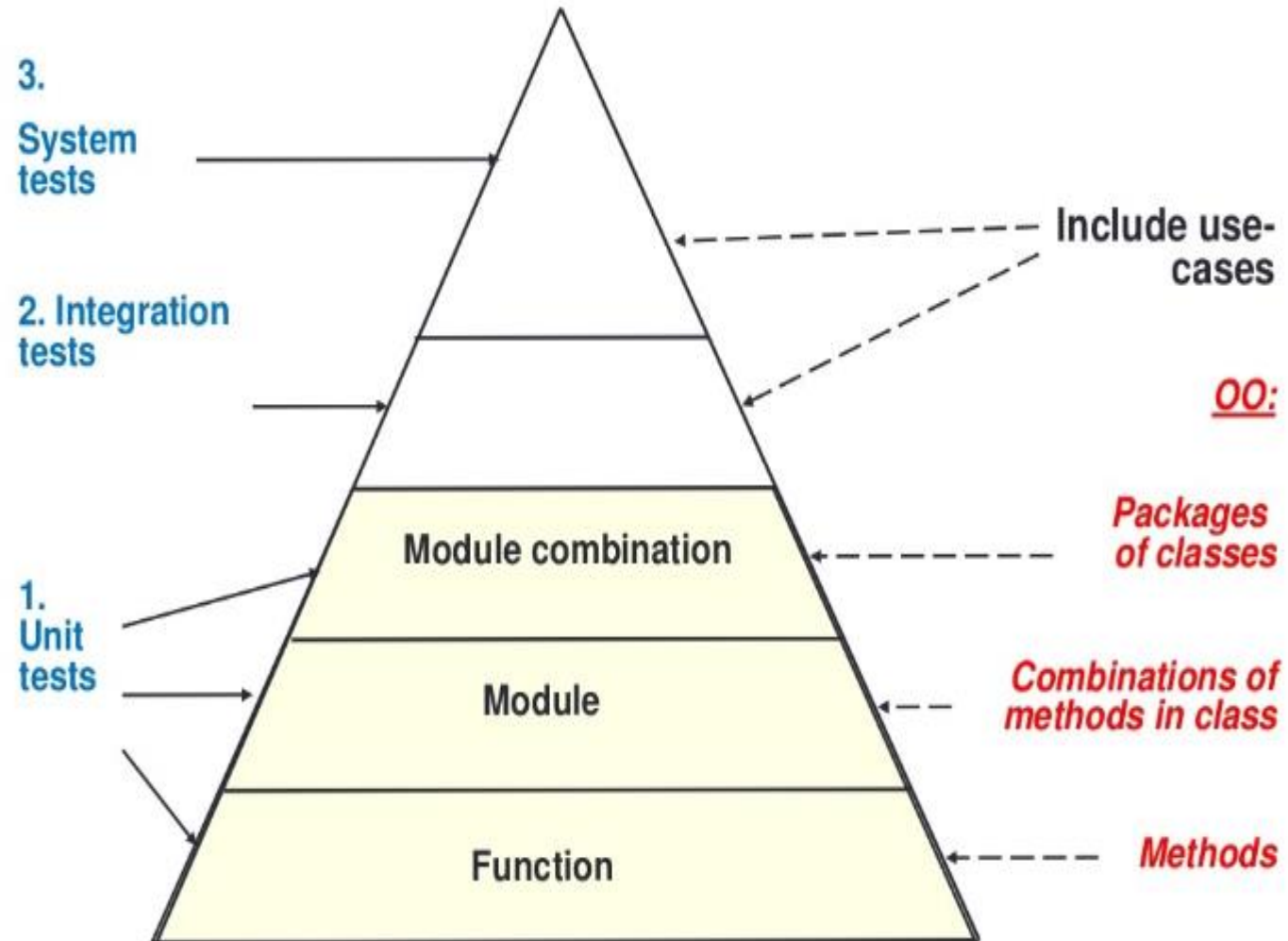
- A unit test is a piece of code that invokes another piece of code and checks the correctness of some assumptions afterward.
- If the assumptions turn out to be un-true,
 - the unit test has failed

```
[TestMethod]
public void MyAddTest()
{
    MyMath m = new MyMath();
    Assert.AreEqual(m.Add(2, 3), 5);
}
```

Testing: the Big Picture



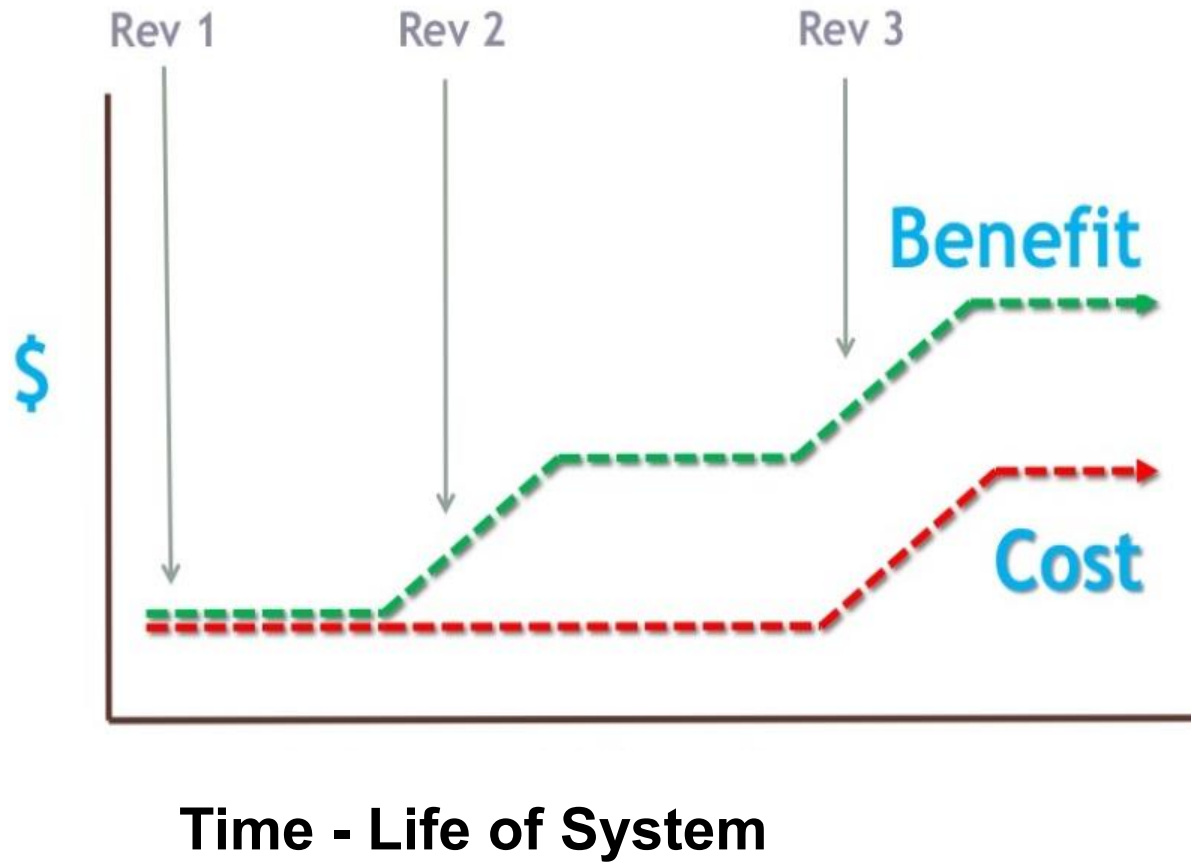
Testing: the Big Picture



Benefits Of Unit Testing

- Safer refactoring
- Smaller, tighter, decoupled code
- Documentation of requirements
- Continuous integration
- Value of tests increase over time

Benefits Of Unit Testing



Unit Testing

- Goal

- Isolate each part of the program and show that the individual parts are correct.

- Benefits

- A unit test provides a strict, written contract that the piece of code must satisfy. As a result, it affords several benefits. Unit tests find problems early in the development cycle

- Limitations

- Testing cannot be expected to catch every error in the program

Elements of a Good Unit Test

- **Automatic**
 - Invocation
 - Checking
- **Thorough**
 - Depends on needs of project
- **Repeatable**
- **Independent**
- **Professional**
 - Test code is "real code"

What is a Unit Test Framework

- Automatically run a set of Unit Tests
- Collects Unit Tests into cohesive groups (e.g. by module)
- Collects Unit Test results into a consolidated report
- Provides setup and teardown support to unit tests

Sample Unit code

```
using System;
namespace MyApp
{
    public class MyMath
    {
        public int Add(int i, int j)
        {
            return i + j;
        }
    }
}
```

Unit Test Frameworks for All

- **Java** JUnit, Cactus, EasyMock
- **Ruby** Test:Unit, RSpec
- **.Net** Nunit, TestDriven, NMock
- **JavaScript** JSUnit, JSpec,
- **Python** PyUnit

Common Unit Test Patterns

- Correct results
- Boundary conditions correct
- Inverse relationships checked (e.g. use result to check itself)
- Cross check results by other means
- Force error condition to occur
- Was performance within bounds (e.g. impact of data set size, network latency)

Common Unit Test Patterns

- **Boundary Conditions:**

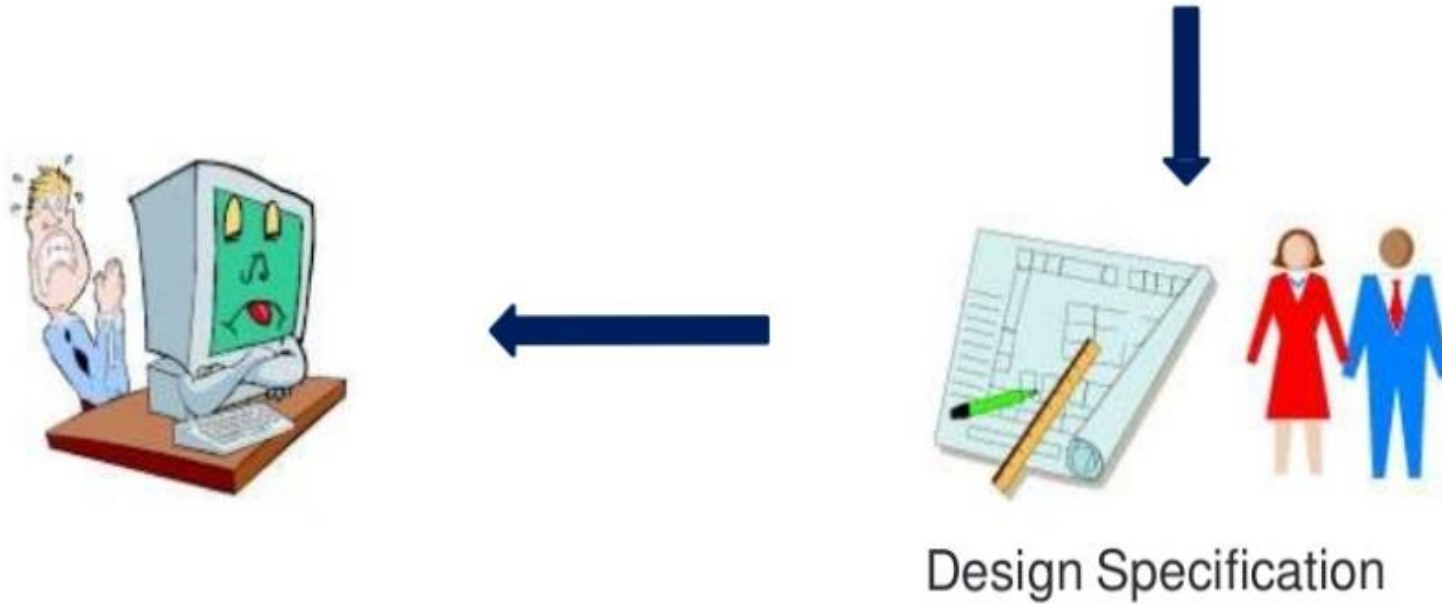
- Format conformance
- Result ordering
- Input data range
- External references required state
- Resource existence (e.g database, file)
- Correct number items (e.g. 0, 1, more than 1)
- Time (e.g. timeout length, tolerance of delay)

Limitations of Unit Testing

- Don't expect Unit Testing to solve all problems or catch all defects - not silver bullet
- Will not catch integration errors - focuses on individual units
- Only 1 part of an effective testing methodology - cover all bases
- Requires rigorous discipline
- Requires sustainable process of construction and remediation

Unit & Integration Testing

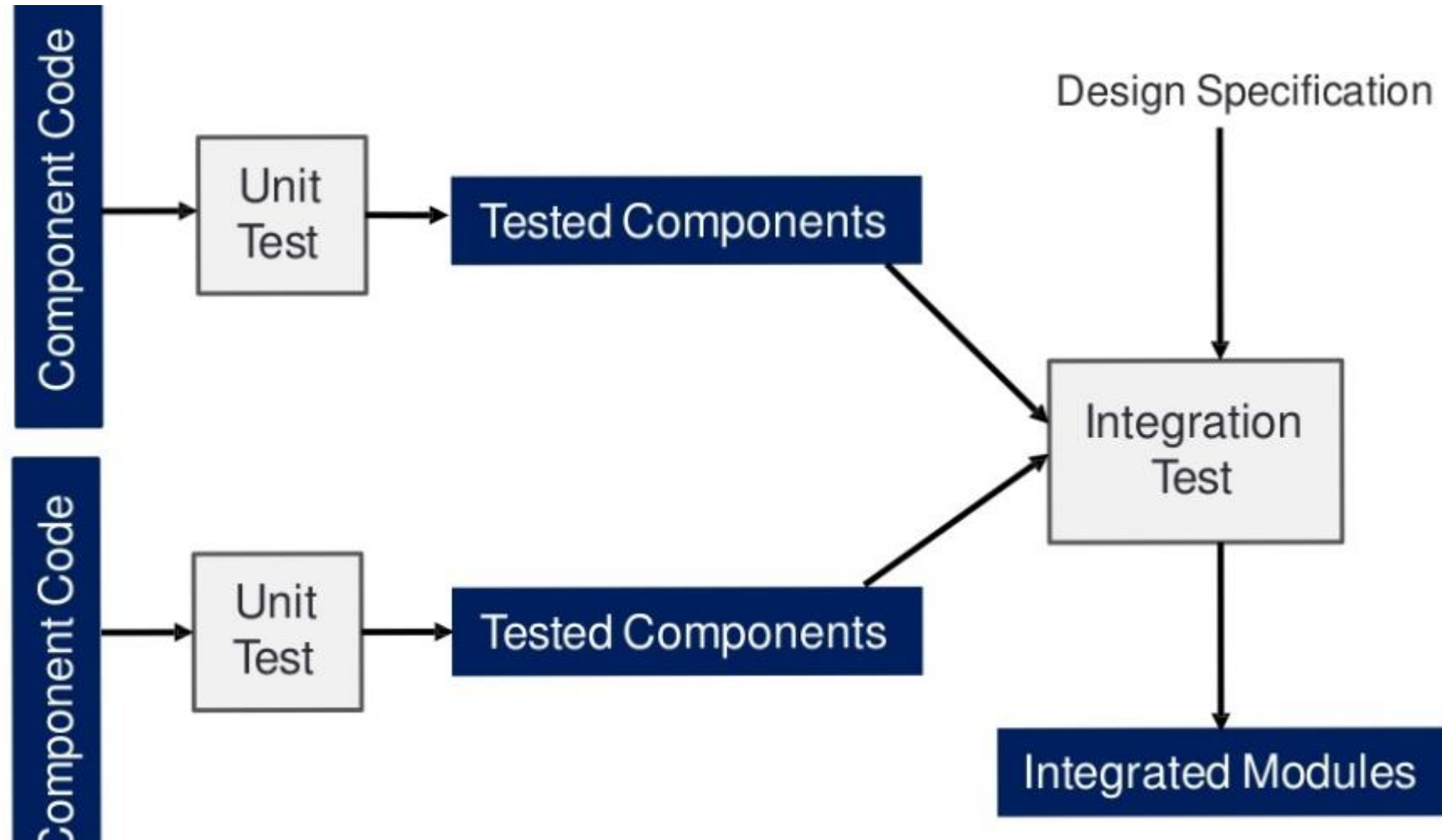
- Objective: To ensure that code implemented the design properly



Unit Testing

- Code Inspections
- Code Walkthroughs
- Black-box Testing
- White-box Testing

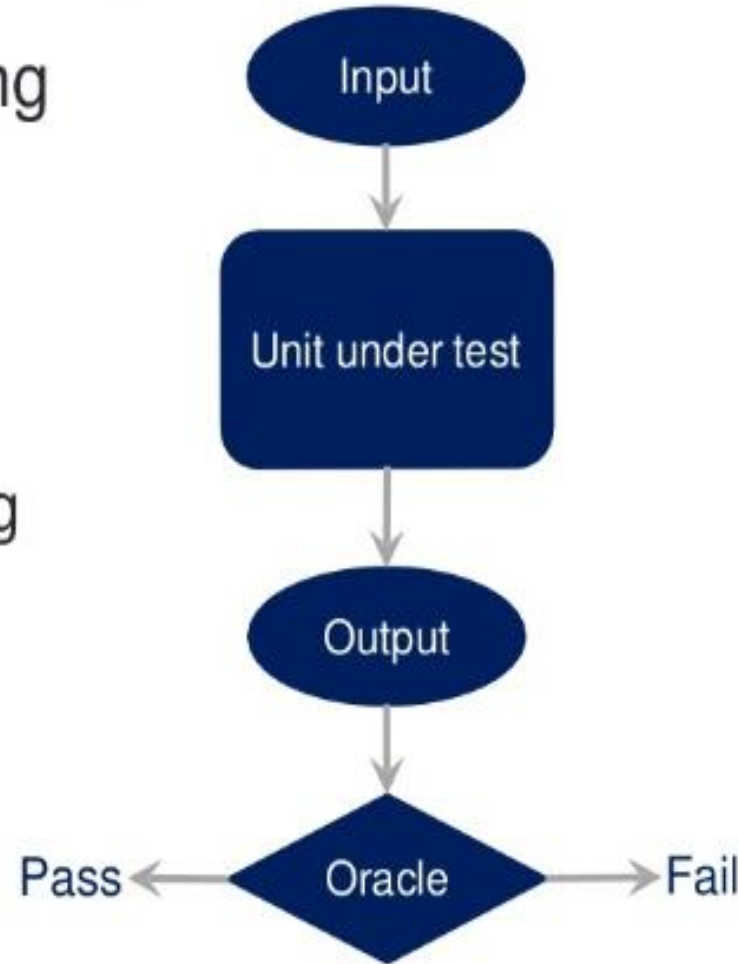
Unit & Integration Testing



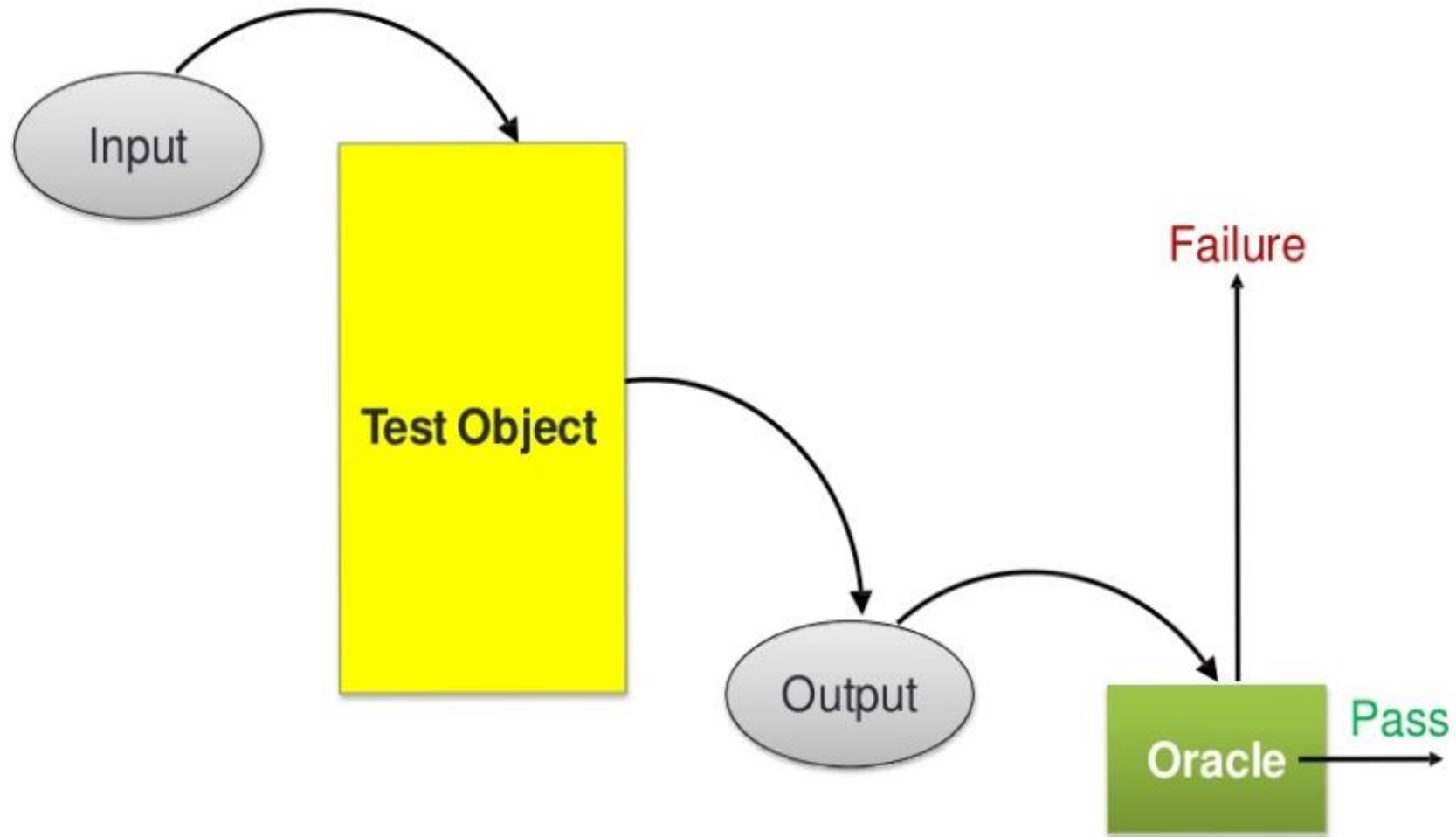
Unit Testing

Different kinds of unit testing

- Code **inspections**
- Code **walkthroughs**
- **Black-box** testing
 - Equivalence class testing
 - Boundary value testing
 - Fuzz testing
- **White-box** testing
 - Coverage-based testing

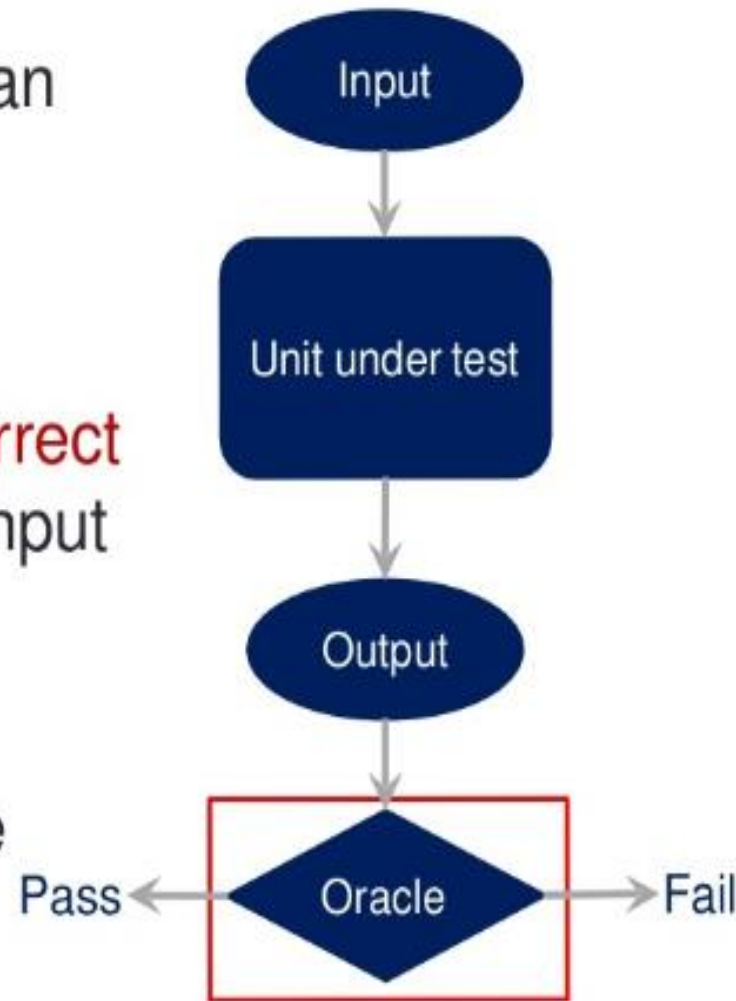


Oracle



Oracles

- **Human:** An **expert** that can examine an **input** and its associated **output** and **determine** whether the program delivered the **correct output** for this particular input
- **Automated:** A **system** capable of performing the above task



Code Inspections



Code Inspections

- **Inspection:** is used to verify the compliance of the product with specified standards and requirements.
- It is done by examining, comparing the product with the designs, code, artefacts and any other documentation available.
- It needs proper planning and overviews are done on the planning to ensure that inspections are held properly.
- Lots of preparations are needed, meetings are held to do inspections and then on the basis of the feedback of the inspection, rework is done.

Code Inspections

- **Code inspection**
 - Check code and documentation against list of concerns
 - Review correctness and efficiency of algorithms
 - Check comments for completeness
 - Formalized process

Goals of Code Inspections

- Identify defects
- Improve code maintainability
- Ensure conformance
 - To design
 - To coding standards
 - To security policies

Code Inspection Roles

- Moderator - 1
- Author(s) - 1 or more
- Reader -1
- Recorder / scribe - 1
- Inspector(s) - multiple

Benefits of Code Inspections

- Improved efficiency at defect removal
- More readable code
- Mentoring / training opportunities
- Learn strengths / weaknesses of individual developers
- Ensure policy conformance

Code walkthrough



Code Walkthrough

- **Walkthroughs:** Author presents their developed artifact to an audience of peers.
- Peers question and comment on the artifact to identify as many defects as possible.
- It involves no prior preparation by the audience. Usually involves minimal documentation of either the process or any arising issues.
- Defect tracking in walkthroughs is inconsistent.

Code Walkthrough

- Code Walkthrough

- Present code and documentation to review team
- Team comments on correctness
- Focus is on the code not the coder
- No influence on developer performance

Code inspection VS. Code walkthrough

Inspection	Walkthrough
Formal	Informal
Initiated by the project team	Initiated by the author
Planned meeting with fixed roles assigned to all the members involved	Unplanned.
Reader reads the product code. Everyone inspects it and comes up with defects.	Author reads the product code and his team mate comes up with defects or suggestions
Recorder records the defects	Author makes a note of defects and suggestions offered by team mate
Moderator has a role in making sure that the discussions proceed on the productive lines	Informal, so there is no moderator

Steps of inspection

- **Planning:**

- choose the inspection team
- schedule the following steps

- **Preparation**

- author collects all artifacts to be inspected
- inspector get familiar with background knowledge

- **Meeting**

- defects are pointed out and recorded

- **Rework**

- author makes changes to the artifacts in response to defects

- **Follow-up**

- moderator verifies the corrections and finish inspection summary report